SyrenBridge

Technical Reference — All Dialects

Analyzer · Transpiler · Architecture · Conversion Coverage

36	13	100%
Analyzer Sources	Transpiler Dialects	No-install Deployment

Syren Cloud · Powered by Databricks Labs Lakebridge · 2026

Table of Contents

1. Platform Overview

SyrenBridge is a Streamlit application deployed on Databricks Apps. It wraps Databricks Labs Lakebridge and extends it with three custom-built migration engines for HiveSQL, Oozie workflows, and SSRS reports. The platform provides a unified UI for analyzing legacy data platform complexity and transpiling source artifacts to Databricks-native formats.

Component	Role	Coverage
Analyzer Tab	Complexity assessment, effort estimation, migration readiness scoring	36 source technologies
Transpiler Tab	Automated code conversion to PySpark, SparkSQL, or Databricks Workflow JSON	13 dialects
Lakebridge CLI	Backs 10 transpiler dialects via BladeBridge	DataStage, Informatica, MSSQL, Netezza, Oracle, Snowflake, SSIS, Synapse, Teradata, Informatica Cloud
HiveSQL Engine	In-process sqlglot conversion (Hive→Databricks SQL), LLM-assisted fix	.hql, .hive, .sql, .ddl, .dml
Oozie Engine	lxml-based workflow/coordinator parser → Databricks Jobs API 2.1 JSON	workflow.xml, coordinator.xml
SSRS Engine	lxml RDL parser → SQL notebooks + assessment JSON, auto-convertibility scoring	.rdl, .rdlc, .rsd

Execution path decision:

- dialect_info.get('oozie') → run_oozie_converter() (lxml engine, no CLI)
- dialect_info.get('ssrs') → run_ssrs_converter() (ssrs_converter engine, no CLI)
- dialect_info.get('custom') → run_hive_transpiler() (sqlglot engine, no CLI)
- All other dialects → run_transpiler() → lakebridge transpile CLI (BladeBridge)

2. Analyzer — All 36 Source Technologies

The Analyzer tab invokes *lakebridge analyze* against uploaded or workspace-browsed files. It produces a complexity report per source: object counts, migration effort score, warnings, and a downloadable assessment ZIP. Files can be uploaded directly or browsed from a Databricks workspace.

#	Technology	Category	File Extensions
1	**ABInitio**	ETL	`.mp .mf .ab .xml .sh .ksh`
2	**ADF**	ETL	`.json .xml`
3	**Alteryx**	ETL	`.yxmd .yxwz .yxmc .yxapp`
4	**Athena**	SQL	`.sql .ddl .dml`
5	**BigQuery**	SQL	`.sql .ddl .dml .json`
6	**BODS**	ETL	`.atl .xml .bods`
7	**Cloudera (Impala)**	SQL	`.sql .ddl .dml`
8	**Datastage**	ETL	`.dsx .xml .pjb`
9	**Greenplum**	SQL	`.sql .ddl .dml`
10	**Hive**	SQL	`.hql .hive .sql .ddl .dml`
11	**IBM DB2**	SQL	`.sql .ddl .dml`
12	**Informatica - Big Data Edition**	ETL	`.xml .session .wf .m .mplt .lkp`
13	**Informatica - PC**	ETL	`.xml .session .wf .m .mplt .lkp .pc`
14	**Informatica Cloud**	ETL	`.xml .json .session`
15	**Jupyter Notebook**	Code	`.ipynb`
16	**MS SQL Server**	SQL	`.sql .ddl .dml .proc .view`
17	**Netezza**	SQL	`.sql .ddl .dml .nzb`
18	**Oozie**	ETL	`.xml .properties .sh .ksh`
19	**Oracle**	SQL	`.sql .ddl .dml .pls .pks .pkb .prc .fnc .vw .trg`
20	**Oracle Data Integrator**	ETL	`.xml .odixml .sql`
21	**PentahoDI**	ETL	`.ktr .kjb .xml`
22	**PIG**	Code	`.pig .txt`
23	**Presto**	SQL	`.sql .ddl .dml`
24	**PySpark**	Code	`.py .ipynb .scala .java`
25	**Redshift**	SQL	`.sql .ddl .dml`

26	**SAPHANA - CalcViews**	SQL	`.xml .hdbcalcview .hdbprocedure .hdbview .analyticview .attributeview`
27	**SAS**	Code	`.sas .sas7bdat`
28	**Snowflake**	SQL	`.sql .ddl .dml`
29	**SPSS**	Code	`.sps .spv`
30	**SQOOP**	ETL	`.xml .sh .ksh .properties`
31	**SSIS**	ETL	`.dtsx .xml`
32	**SSRS**	ETL	`.rdl .rdlc .rsd`
33	**Synapse**	SQL	`.sql .ddl .dml .json`
34	**Talend**	ETL	`.item .properties .xml .java`
35	**Teradata**	SQL	`.sql .bteq .tdl .tpt .ddl .dml`
36	**Vertica**	SQL	`.sql .ddl .dml`

2.1 Analyzer — Per-Technology Detail

1. ABInitio

ETL `.mp .mf .ab .xml .sh .ksh`

Metadata-driven ETL workloads built on Ab Initio GDE/Co>Op. Assessment covers graph layouts, component types, and partitioning strategies.

---

2. ADF

ETL `.json .xml`

Azure Data Factory pipelines in ARM/JSON format. Assessment covers activity types, linked services, triggers, and parameter passing.

---

3. Alteryx

ETL `.yxmd .yxwz .yxmc .yxapp`

Alteryx Designer workflows (.yxmd/.yxwz). Assessment maps tool types (Join, Formula, Summarize) to Spark/Python equivalents.

---

4. Athena

SQL `.sql .ddl .dml`

AWS Athena SQL (Presto-based). Assessment flags ANSI gaps, partition projections, and S3 references that need remapping to Delta Lake.

---

5. BigQuery

SQL `.sql .ddl .dml .json`

Google BigQuery SQL. Assessment flags ARRAY/STRUCT types, UNNEST patterns, and BigQuery-specific functions needing Spark equivalents.

---

6. BODS

ETL `.atl .xml .bods`

SAP BusinessObjects Data Services. Assessment covers dataflows, transforms, and lookup objects in ATL/XML format.

---

7. Cloudera (Impala)

SQL `.sql .ddl .dml`

Impala SQL on Cloudera CDH/CDP. Assessment identifies Kudu-specific syntax, COMPUTE STATS calls, and HDFS-rooted data paths.

---

8. Datastage

ETL `.dsx .xml .pjb`

IBM DataStage jobs exported as .dsx or .pjb. Assessment covers stage types (Transformer, Join, Aggregator) and runtime parameters.

---

9. Greenplum

SQL `.sql .ddl .dml`

Greenplum PostgreSQL-based SQL. Assessment flags distribution keys, append-optimised tables, and external table definitions.

---

10. Hive

SQL `.hql .hive .sql .ddl .dml`

HiveQL from Cloudera/Hortonworks. Assessment identifies STORED AS clauses, SerDe configurations, partitioned inserts, and dynamic partition settings.

---

11. IBM DB2

SQL `.sql .ddl .dml`

IBM DB2 LUW / z/OS SQL. Assessment covers REXX-style syntax, DB2-specific date arithmetic, and catalog object references.

---

12. Informatica - Big Data Edition

ETL `.xml .session .wf .m .mplt .lkp`

Informatica BDE (native Hadoop) mappings. Assessment covers pushdown optimisation settings, HDFS sources, and Spark-native equivalents.

---

13. Informatica - PC

ETL `.xml .session .wf .m .mplt .lkp .pc`

Informatica PowerCenter mappings, sessions, and workflows. Assessment maps source/target definitions, transformations, and session parameters.

---

14. Informatica Cloud

ETL `.xml .json .session`

Informatica IICS/CDI taskflows. Assessment covers connection references, mapping tasks, schedule triggers, and cloud connector types.

---

15. Jupyter Notebook

Code `.ipynb`

Jupyter notebooks (.ipynb). Assessment identifies Spark/pandas API patterns, magic commands, and cells with side-effects needing review.

---

16. MS SQL Server

SQL `.sql .ddl .dml .proc .view`

T-SQL: stored procedures, views, functions, DDL. Assessment flags T-SQL-specific constructs: MERGE, TOP N, NOLOCK hints, CONVERT, GETDATE, cursors.

---

17. Netezza

SQL `.sql .ddl .dml .nzb`

IBM Netezza SQL including SPU-specific syntax. Assessment covers GROOM TABLE, distribution keys, zone maps, and Netezza UDFs.

---

18. Oozie

ETL `.xml .properties .sh .ksh`

Apache Oozie workflow.xml and coordinator.xml files. Assessment maps actions (Hive, Pig, Shell, Java, Spark) and coordinator scheduling.

---

19. Oracle

SQL `.sql .ddl .dml .pls .pks .pkb .prc .fnc .vw .trg`

Oracle SQL and PL/SQL: packages, procedures, functions, triggers. Assessment flags Oracle-specific syntax: CONNECT BY, ROWNUM, NVL, dbms_* packages.

---

20. Oracle Data Integrator

ETL `.xml .odixml .sql`

ODI interfaces and knowledge modules in XML export format. Assessment covers IKM/LKM types, staging areas, and CDC configurations.

---

21. PentahoDI

ETL `.ktr .kjb .xml`

Pentaho Kettle (.ktr transformations, .kjb jobs). Assessment covers step types, hop conditions, and job entry parameters.

---

22. PIG

Code `.pig .txt`

Apache Pig Latin scripts (.pig). Assessment maps LOAD/STORE paths, relation aliases, UDFs, and GROUP/JOIN operations to Spark.

---

23. Presto

SQL `.sql .ddl .dml`

Presto/Trino SQL. Assessment flags connector-specific syntax, Lambda functions, and WITH ORDINALITY clauses.

---

24. PySpark

Code `.py .ipynb .scala .java`

Spark Classic (PySpark, Scala Spark, Java Spark). Assessment identifies APIs deprecated in Spark 3.x/4.x and patterns incompatible with Serverless.

---

25. Redshift

SQL `.sql .ddl .dml`

Amazon Redshift SQL. Assessment flags DISTKEY/SORTKEY DDL, UNLOAD/COPY commands, and Redshift-specific window functions.

---

26. SAPHANA - CalcViews

SQL `.xml .hdbcalcview .hdbprocedure .hdbview .analyticview .attributeview`

SAP HANA Calculation Views and procedures in .hdbcalcview/.xml format. Assessment covers join types, filter nodes, and aggregation behaviour.

---

27. SAS

Code `.sas .sas7bdat`

SAS Base and SAS Macro language. Assessment flags DATA step patterns, PROC SQL usage, macro variables, and SAS library references.

---

28. Snowflake

SQL `.sql .ddl .dml`

Snowflake SQL. Assessment flags VARIANT/ARRAY/OBJECT types, FLATTEN, Snowflake JavaScript UDFs, and warehouse-size references.

---

29. SPSS

Code `.sps .spv`

IBM SPSS Modeler streams (.sps syntax files). Assessment covers statistical procedure calls and data file references.

---

30. SQOOP

ETL `.xml .sh .ksh .properties`

Apache Sqoop import/export jobs in shell scripts or XML configs. Assessment maps JDBC connection params and split-by column patterns.

---

31. SSIS

ETL `.dtsx .xml`

SQL Server Integration Services packages (.dtsx). Assessment covers Data Flow Tasks, Execute SQL Tasks, and Control Flow complexity.

---

32. SSRS

ETL `.rdl .rdlc .rsd`

SQL Server Reporting Services reports (.rdl). Assessment classifies datasets (Text vs StoredProcedure), flags VB.NET code blocks, and rates auto-convertibility.

---

33. Synapse

SQL `.sql .ddl .dml .json`

Azure Synapse Analytics SQL pools. Assessment flags Synapse-specific DDL (DISTRIBUTION, PARTITION), PolyBase EXTERNAL TABLE definitions, and linked service references.

---

34. Talend

ETL `.item .properties .xml .java`

Talend Open Studio jobs in .item/.properties format. Assessment covers component types (tMap, tJDBCInput, tFileInput) and Java subjobs.

---

35. Teradata

SQL `.sql .bteq .tdl .tpt .ddl .dml`

Teradata SQL and BTEQ scripts. Assessment flags Teradata-specific syntax: QUALIFY, NORMALIZE, COMPRESS, SET vs MULTISET tables, and volatile tables.

---

36. Vertica

SQL `.sql .ddl .dml`

Vertica SQL including PROJECTIONS, SEGMENTED BY, and Vertica-specific UDFs. Assessment covers projection DDL and Vertica analytic functions.

---

3. Transpiler — All 13 Dialects

The Transpiler tab converts source artifacts to Databricks-native code. Each dialect has a dedicated engine (Lakebridge CLI or built-in), fixed or selectable output format, and a clearly defined conversion scope. The sections below provide per-dialect detail.

Dialect	Engine	Output	Extensions
DataStage	Lakebridge CLI (BladeBridge)	PySpark / SparkSQL	`.dsx .xml .pjb`
HiveSQL (Cloudera)	Built-in engine (sqlglot)	PySpark / SparkSQL	`.hql .hive .sql .ddl .dml`
Informatica	Lakebridge CLI (BladeBridge)	PySpark / SparkSQL	`.xml .session .wf .m .mplt .lkp`
Informatica Cloud	Lakebridge CLI (BladeBridge)	PySpark / SparkSQL	`.xml .json .session`
MS SQL Server	Lakebridge CLI (BladeBridge)	PySpark / SparkSQL	`.sql .ddl .dml .proc .view`
Netezza	Lakebridge CLI (BladeBridge)	PySpark / SparkSQL	`.sql .ddl .dml .nzb`
Oracle	Lakebridge CLI (BladeBridge)	PySpark / SparkSQL	`.sql .ddl .dml .pls .pks .pkb .prc .fnc .vw .trg`
Snowflake	Lakebridge CLI (BladeBridge)	PySpark / SparkSQL	`.sql .ddl .dml`
SSIS	Lakebridge CLI — BladeBridge (SparkSQL only)	SparkSQL	`.dtsx .xml`
SSRS (Reports)	Built-in engine (ssrs_converter) — no CLI required	SQL Notebooks + Assessment JSON	`.rdl .rdlc .rsd`
Synapse	Lakebridge CLI (BladeBridge)	PySpark / SparkSQL	`.sql .ddl .dml .json`
Teradata	Lakebridge CLI (BladeBridge)	PySpark / SparkSQL	`.sql .bteq .tdl .tpt .ddl .dml`
Oozie (Workflow)	Built-in engine (oozie_converter) — no CLI required	Databricks Workflow JSON	`.xml`

3.1 DataStage

ETL	Lakebridge CLI (BladeBridge)	Output: PySpark / SparkSQL

Converts IBM DataStage parallel jobs and server jobs exported in DSX or PJB format to Databricks-compatible PySpark notebooks or SparkSQL.

Accepted file types:

```
.dsx .xml .pjb
```

What is automatically converted:

- ✓ Transformer stage → withColumn() / select() expressions
- ✓ Join stage → df.join() with configurable join type
- ✓ Aggregator stage → df.groupBy().agg()
- ✓ Sequential File / DB2 source → spark.read.jdbc() / spark.read.csv()
- ✓ DB2 / Oracle destination → df.write.saveAsTable() / jdbc()
- ✓ Shared containers → extracted as reusable PySpark functions
- ✓ Job sequences and sequencer stages → Databricks Workflow tasks

What requires manual migration:

- ■ Custom C/C++ transforms — rewrite as PySpark UDFs
- ■ Before/After SQL subroutines — migrate as separate notebook cells
- ■ DataStage parameters → map to Databricks job parameters or widgets

Output:

Output is a PySpark or SparkSQL notebook per DataStage job, plus a migration summary report.

3.2 HiveSQL (Cloudera)

SQL	Built-in engine (sqlglot)	Output: PySpark / SparkSQL

Custom in-process transpiler using sqlglot (read='hive', write='databricks'). Post-processes output to strip Hive-only DDL clauses and optionally refines complex statements via an LLM endpoint.

Accepted file types:

```
.hql .hive .sql .ddl .dml
```

What is automatically converted:

- ✓ SELECT / INSERT / CREATE TABLE / CTAS statements
- ✓ Hive UDFs referenced by name — preserved with migration comment
- ✓ Partition predicates and dynamic partition inserts
- ✓ LATERAL VIEW EXPLODE → Spark LATERAL VIEW equivalent
- ✓ Subqueries and CTEs

✓ Hive date functions → Spark SQL equivalents via sqlglot

What requires manual migration:

- STORED AS / ROW FORMAT / SERDE / TBLPROPERTIES / LOCATION hdfs:// — stripped with comment
- Custom Hive UDFs — must be registered as Spark UDFs manually
- Hive authorization (GRANT/REVOKE on table level) — migrate to Unity Catalog grants

Output:

Output is Databricks SQL dialect (runs on SQL Warehouses). LLM-assisted fix applied per statement when LLM endpoint is configured.

3.3 Informatica

ETL	Lakebridge CLI (BladeBridge)	Output: PySpark / SparkSQL

Converts Informatica PowerCenter mappings, sessions, and workflows (XML export) to PySpark notebooks. Covers source/target definitions, transformations, and session-level parameters.

Accepted file types:

```
.xml .session .wf .m .mplt .lkp
```

What is automatically converted:

- ✓ Source Qualifier → spark.read.jdbc() with pushdown SQL
- ✓ Expression / Filter / Router transformations → PySpark column ops
- ✓ Aggregator → groupBy().agg()
- ✓ Joiner → df.join()
- ✓ Lookup → broadcast join or lookup DataFrame
- ✓ Sorter → df.orderBy()
- ✓ Normalizer → explode() on repeated fields
- ✓ Update Strategy → merge/insert logic on Delta table

What requires manual migration:

- ■ Custom Java Transformations — rewrite as PySpark UDFs
- ■ Mapping parameters/variables — map to Databricks job parameters
- ■ Workflow scheduling — migrate to Databricks Jobs scheduler

Output:

One PySpark notebook per mapping. Workflow-level orchestration mapped to a Databricks Workflow JSON.

3.4 Informatica Cloud

ETL	Lakebridge CLI (BladeBridge)	Output: PySpark / SparkSQL

Converts Informatica IICS (Intelligent Cloud Services) / CDI (Cloud Data Integration) taskflow exports to PySpark.

Accepted file types:

```
.xml .json .session
```

What is automatically converted:

- ✓ Mapping tasks → PySpark notebooks
- ✓ Connection references (Salesforce, S3, JDBC) → Databricks connection configs
- ✓ Parameterised schedules → Databricks Jobs triggers

What requires manual migration:

- IICS-specific connectors without Databricks equivalent — manual integration
- Data Quality rules embedded in tasks — review for Databricks DQ equivalent

Output:

PySpark notebooks with connection parameter stubs for engineer completion.

3.5 MS SQL Server

SQL	Lakebridge CLI (BladeBridge)	Output: PySpark / SparkSQL

Converts T-SQL objects — stored procedures, views, DDL scripts, DML — to Spark SQL or PySpark. Handles the most common T-SQL constructs including MERGE, TOP N, NOLOCK, and date functions.

Accepted file types:

```
.sql .ddl .dml .proc .view
```

What is automatically converted:

- ✓ SELECT / INSERT / UPDATE / DELETE → Spark SQL / Delta merge
- ✓ CREATE TABLE / CREATE VIEW → Databricks DDL
- ✓ MERGE → Delta Lake MERGE INTO
- ✓ TOP N → LIMIT N
- ✓ GETDATE() / GETUTCDATE() → current_timestamp()
- ✓ ISNULL(a,b) → ifnull(a,b)
- ✓ CONVERT(type, val) → CAST(val AS type)
- ✓ Stored procedures → PySpark functions or SQL notebooks
- ✓ Common Table Expressions (WITH ...) → preserved
- ✓ Window functions (ROW_NUMBER, RANK, LEAD/LAG) → preserved

What requires manual migration:

- ■ Cursors — rewrite as set-based Spark operations
- ■ Dynamic SQL (EXEC sp_executesql) — requires manual analysis
- ■ NOLOCK hints → removed (Delta Lake handles MVCC)
- ■ Linked server references — must be remapped to Databricks external connections
- ■ CLR stored procedures — rewrite as Python UDFs

Output:

One SQL notebook or PySpark file per source script. Warnings generated for each pattern requiring manual review.

3.6 Netezza

SQL	Lakebridge CLI (BladeBridge)	Output: PySpark / SparkSQL

Converts IBM Netezza (now IBM Db2 Warehouse) SQL to Spark SQL. Handles Netezza's MPP-specific DDL including distribution and zone map syntax.

Accepted file types:

```
.sql .ddl .dml .nzb
```

What is automatically converted:

- ✓ SELECT / INSERT / CREATE TABLE → standard Spark SQL
- ✓ DISTRIBUTE ON → stripped (Delta Lake manages layout)
- ✓ ORGANIZE ON → converted to ZORDER BY hint comment
- ✓ Netezza string / date functions → Spark equivalents
- ✓ NZPLSQL procedures → PySpark functions
- ✓ External tables (NZB format references) → flagged with migration notes

What requires manual migration:

- ■ NZB backup files — extract SQL DDL via Netezza tooling first
- ■ GROOM TABLE — no direct equivalent; use OPTIMIZE on Delta tables
- ■ Netezza UDFs (C-based) — rewrite as Python UDFs

Output:

SparkSQL notebooks with distribution clauses removed and ZORDER hints added as comments.

3.7 Oracle

SQL	Lakebridge CLI (BladeBridge)	Output: PySpark / SparkSQL

Converts Oracle SQL and PL/SQL — packages, procedures, functions, triggers — to Spark SQL or PySpark. One of the most comprehensive conversion paths in Lakebridge.

Accepted file types:

`.sql .ddl .dml .pls .pks .pkb .prc .fnc .vw .trg`

What is automatically converted:

- ✓ SELECT / INSERT / MERGE / CREATE TABLE / CREATE VIEW
- ✓ CONNECT BY hierarchical queries → WITH RECURSIVE CTE
- ✓ ROWNUM → ROW_NUMBER() OVER () or LIMIT
- ✓ NVL(a,b) → ifnull(a,b), NVL2 → CASE WHEN
- ✓ DECODE → CASE WHEN
- ✓ SYSDATE → current_date(), SYSTIMESTAMP → current_timestamp()
- ✓ TO_DATE / TO_CHAR → date_format() / to_date()
- ✓ Sequences (NEXTVAL/CURRVAL) → Databricks IDENTITY columns
- ✓ ROWID references → flagged for removal
- ✓ PL/SQL anonymous blocks → PySpark cells
- ✓ Package specs/bodies → Python modules

What requires manual migration:

- ■ DBMS_* packages — replace with Python/Databricks API equivalents
- ■ UTL_FILE, UTL_HTTP — rewrite using Python file/HTTP libs
- ■ Triggers — no direct Delta equivalent; consider CDC or DLT
- ■ Global Temporary Tables → Spark temp views or session caches
- ■ Oracle spatial (SDO_*) — map to Databricks Mosaic or PostGIS

Output:

High-confidence auto-conversion for standard SQL. PL/SQL bodies emitted as PySpark with manual flags for complex constructs.

3.8 Snowflake

SQL	Lakebridge CLI (BladeBridge)	Output: PySpark / SparkSQL

Converts Snowflake SQL to Spark SQL. Modern SQL dialects are close; key differences are Snowflake semi-structured types and JavaScript UDFs.

Accepted file types:

`.sql .ddl .dml`

What is automatically converted:

- ✓ Standard SELECT / DDL / DML → Spark SQL
- ✓ FLATTEN / LATERAL FLATTEN → LATERAL VIEW EXPLODE
- ✓ PARSE_JSON / TO_VARIANT → from_json() / to_json()
- ✓ ARRAY_AGG → collect_list()
- ✓ OBJECT_CONSTRUCT → named_struct() or map()
- ✓ Snowflake date/time functions → Spark equivalents
- ✓ QUALIFY clause → wrapped in outer SELECT with WHERE on window alias
- ✓ Streams and Tasks references → flagged for Databricks DLT / Jobs

What requires manual migration:

- ■ JavaScript UDFs — rewrite as Python UDFs
- ■ Snowflake Streams / Tasks — migrate to DLT pipelines or Databricks Jobs
- ■ External stages (S3/Azure/GCS references) — remap to Unity Catalog volumes
- ■ Snowpark Python code — review for PySpark compatibility

Output:

SparkSQL output with semi-structured type conversions annotated for engineer review.

3.9 SSIS

ETL	Lakebridge CLI — BladeBridge (SparkSQL only)	Output: SparkSQL

Converts SQL Server Integration Services packages (.dtsx) to SparkSQL via Databricks Labs BladeBridge. Output format is fixed to SparkSQL.

Accepted file types:

`.dtsx .xml`

What is automatically converted:

- ✓ OLE DB Source (SQL query) → spark.sql() read cell
- ✓ OLE DB Destination → df.write.saveAsTable()
- ✓ Derived Column → df.withColumn() with column expressions
- ✓ Conditional Split → df.filter() into separate DataFrames
- ✓ Execute SQL Task (MERGE) → Delta Lake MERGE INTO
- ✓ Lookup → broadcast join
- ✓ Aggregate transform → groupBy().agg()
- ✓ Sort → df.orderBy()
- ✓ Precedence constraints → Databricks Workflow task dependencies

What requires manual migration:

- ■ Script Tasks / Script Components (C# or VB.NET) — rewrite as PySpark UDFs
- ■ Custom third-party SSIS components — no automatic mapping
- ■ Complex @[User::Variable] expressions — may need manual adjustment
- ■ FTP / SMTP / WMI tasks — no Spark equivalent; use Python libraries
- ■ Event handlers — not converted; implement as Databricks Jobs notifications

Output:

SparkSQL notebook(s) per DTSX package. BladeBridge limitation: only SparkSQL target is supported for SSIS; PySpark output is not available.

3.10 SSRS (Reports)

Reporting	Built-in engine (ssrs_converter) — no CLI required	Output: SQL Notebooks + Assessment JSON

Custom SSRS-to-Databricks converter built into SyrenBridge. Parses RDL XML with lxml, classifies each report's convertibility, generates a SQL notebook per auto-convertible report, and produces a structured assessment JSON for every report.

Accepted file types:

`.rdl .rdlc .rsd`

What is automatically converted:

- ✓ Text-query datasets → one SQL cell per dataset in a .sql notebook
- ✓ Parameters → -- DECLARE comments (replace with dbutils.widgets)
- ✓ TableDirect datasets → SELECT * FROM
- ✓ T-SQL function flags: GETDATE→current_timestamp, ISNULL→ifnull, TOP N→LIMIT N, DATEADD, DATEDIFF, CONVERT, NOLOCK
- ✓ Assessment JSON: data sources, datasets, report items (Tablix/Chart/Matrix), parameters, VB.NET code

What requires manual migration:

- ■ StoredProcedure datasets — migrate the proc logic, then add SELECT query
- ■ VB.NET code blocks — rewrite as Python UDFs or SQL CASE expressions
- ■ Report layout, formatting, charts — use Databricks Dashboards (Lakeview) or a BI tool
- ■ Parameters → replace -- DECLARE stubs with dbutils.widgets.get() calls
- ■ Cross-report drillthrough links — no automated equivalent

Output:

One .sql notebook + one _assessment.json per RDL file. Non-auto-convertible reports produce assessment JSON only (no notebook).

3.11 Synapse

SQL	Lakebridge CLI (BladeBridge)	Output: PySpark / SparkSQL

Converts Azure Synapse Analytics dedicated SQL pool scripts to Spark SQL. Handles Synapse MPP DDL and PolyBase external table patterns.

Accepted file types:

```
.sql .ddl .dml .json
```

What is automatically converted:

- ✓ CREATE TABLE with DISTRIBUTION and PARTITION → Databricks DDL with USING DELTA
- ✓ DISTRIBUTION = HASH(col) → noted in COMMENT (Delta handles layout)
- ✓ PolyBase EXTERNAL TABLE / CREATE EXTERNAL DATA SOURCE → Unity Catalog external table stub
- ✓ Standard SELECT / INSERT / MERGE → Spark SQL
- ✓ Synapse-specific functions → Spark equivalents
- ✓ Synapse Pipelines JSON (ARM) → Databricks Workflow tasks

What requires manual migration:

- ■ Serverless SQL pool queries over Azure Data Lake → remap to Unity Catalog volumes
- ■ PolyBase credential objects — replace with Databricks secret scopes
- ■ STATISTICS / RESULT_SET_CACHING — no direct equivalent
- ■ Workload management (WORKLOAD GROUP) — use Databricks cluster policies

Output:

DDL clauses specific to Synapse MPP (DISTRIBUTION, STATISTICS) are stripped with comments; output runs directly on Databricks SQL Warehouse.

3.12 Teradata

SQL	Lakebridge CLI (BladeBridge)	Output: PySpark / SparkSQL

Converts Teradata SQL, BTEQ scripts, and TPT (Teradata Parallel Transporter) files. One of the most common enterprise DW-to-Databricks migrations.

Accepted file types:

```
.sql .bteq .tdl .tpt .ddl .dml
```

What is automatically converted:

- ✓ SELECT / INSERT / CREATE TABLE (SET/MULTISET) → Spark SQL
- ✓ QUALIFY → ROW_NUMBER() OVER () in outer SELECT
- ✓ NORMALIZE → Spark window function equivalent
- ✓ COMPRESS (column compression) → stripped (Delta handles encoding)

- ✓ VOLATILE TABLE → Spark temp view or CREATE TEMP TABLE
- ✓ PRIMARY INDEX → noted in COMMENT (Delta auto-manages layout)
- ✓ BTEQ control flow (.IF / .GOTO / .QUIT) → Python conditional notebook cells
- ✓ TPT APPLY ... OPERATOR patterns → PySpark bulk load equivalent
- ✓ Teradata date arithmetic (DATE + 1) → date_add()
- ✓ OREPLACE → regexp_replace()
- ✓ ZEROIFNULL / NULLIFZERO → ifnull(col,0) / nullif(col,0)

What requires manual migration:

- ■ Teradata stored procedures (SPL) — rewrite as PySpark functions
- ■ Macro objects → PySpark functions with parameter injection
- ■ COLLECT STATISTICS — no equivalent; use ANALYZE TABLE on Delta
- ■ Multi-statement requests (MSR) — split into individual cells
- ■ FastLoad / MultiLoad scripts → Databricks batch ingest patterns

Output:

BTEQ scripts are split into SQL cells; control flow converted to Python. High conversion coverage for standard DW SQL patterns.

3.13 Oozie (Workflow)

ETL	Built-in engine (oozie_converter) — no CLI required	Output: Databricks Workflow JSON

Custom Oozie-to-Databricks converter built into SyrenBridge. Parses workflow.xml and coordinator.xml files with lxml and outputs Databricks Jobs API 2.1 JSON, ready for direct deployment via /api/2.1/jobs.

Accepted file types:

`.xml`

What is automatically converted:

- ✓ workflow.xml → Databricks Workflow (tasks with dependencies)
- ✓ coordinator.xml → Databricks scheduled Job (Quartz cron → Databricks cron)
- ✓ Hive action → notebook_task pointing to migrated HiveSQL notebook
- ✓ Spark action → spark_python_task or notebook_task
- ✓ Shell action → notebook_task with shell commands as %sh magic
- ✓ Java action → spark_jar_task stub
- ✓ Pig action → notebook_task (Pig Latin converted separately)
- ✓ Fork/Join → parallel tasks with dependency tracking (fan-in DAG)
- ✓ Coordinator→workflow linking → run_job_task with sentinel {{job_id:}}
- ✓ Decision nodes (switch/case) → Databricks conditional task dependencies

What requires manual migration:

- ■ EL expressions (${...}) — preserved verbatim, engineer resolves to Databricks job params
- ■ Datasets and SLA blocks — not converted; document in migration notes
- ■ Coordinator end-time — captured in coordinator_info JSON field only
- ■ Sub-workflow actions → inline tasks or separate workflow job
- ■ Custom action types not in the standard set

Output:

Output is deployable Databricks Workflow JSON. Coordinator→workflow links use a sentinel that the UI auto-replaces with the real job_id after the workflow job is created.

---

4. Architecture & Data Flow

4.1 Deployment

- SyrenBridge is a single Streamlit app deployed as a Databricks App (no additional infrastructure).
- Lakebridge CLI is installed in the app's Python environment — no separate server.
- Custom engines (HiveSQL, Oozie, SSRS) are pure Python modules — no external services.
- Credentials (Databricks host/token) are passed via environment variables or the Settings tab; used for workspace browsing and file upload.

4.2 Module Boundaries

- modules/ — pure Python, no Streamlit imports. Independently testable with pytest.
- app.py — all Streamlit rendering; imports from modules/ only.
- PySpark (sql_validator.py, dummy_data.py) — only used in test/validation path, never in the Streamlit render path.

4.3 File Flow

Step	Action	Location
1	User uploads files or browses Databricks workspace	app.py — Upload Files / Databricks Workspace tabs
2	Files written to a temp directory on the app server	tempfile.mkdtemp()
3	Engine invoked with src_dir and out_dir	run_transpiler / run_hive_transpiler / run_oozie_converter / run_ssrs_converter
4	Engine writes output files to out_dir	Lakebridge CLI writes notebooks; custom engines write .sql/.json/.workflow_json
5	UI reads out_dir and presents file tree, metrics, download ZIP	app.py — results section
6	(Optional) Upload ZIP to Databricks workspace	DatabricksClient.upload_directory_to_workspace()

4.4 HiveSQL Pipeline Detail

- Pre-processing: clean whitespace, detect encoding issues
- Split into statements (line-aware splitter)
- sqlglot.transpile(sql, read='hive', write='databricks') per statement
- Post-processing rules: strip STORED AS, ROW FORMAT, SERDE, TBLPROPERTIES, LOCATION hdfs://
- Issue tagging: flag statements with warnings

- Optional LLM fix: per-statement HTTP call to configured LLM endpoint for complex patterns
- Final output: concatenated Databricks SQL file

4.5 Oozie Converter Detail

- Parse workflow.xml and coordinator.xml with lxml.etree
- Workflow → Databricks Job with tasks (one per Oozie action)
- Fan-in DAG: predecessors tracked as Dict[str, List[str]] (supports multiple upstream edges)
- Cluster rule: job_clusters (shared) only for 2+ task jobs; single-task → new_cluster inline
- Coordinator → run_job_task with sentinel {{job_id:}} after workflow linking
- Coordinator→workflow matching: basename exact, then normalised (-↔_, lowercase)
- _strip_annotation_keys() removes _-prefixed migration metadata from workflow jobs

4.6 SSRS Converter Detail

- Parse .rdl XML with lxml.etree; auto-detect RDL namespace
- Extract: DataSources, ReportParameters, Code (VB.NET), DataSets (CommandType + SQL), ReportItems
- Classify: Text → convertible; StoredProcedure → manual; TableDirect → SELECT * FROM
- Flag T-SQL patterns: GETDATE, ISNULL, TOP N, DATEADD, DATEDIFF, CONVERT, NOLOCK
- auto_convertible = True if at least one Text dataset exists
- Generate .sql notebook (one cell per dataset) + _assessment.json (full structural inventory)

5. Quick Reference

Transpiler Engine Decision Table

Dialect flag in TRANSPILER_DIALECTS	Engine called	Output format
`oozie: True`	run_oozie_converter()	Databricks Workflow JSON
`ssrs: True`	run_ssrs_converter()	SQL notebooks + assessment JSON
`custom: True (HiveSQL)`	run_hive_transpiler()	Databricks SQL (SparkSQL or PySpark)
`sparksql_only: True (SSIS)`	run_transpiler() → lakebridge CLI	SparkSQL only
`(none of the above)`	run_transpiler() → lakebridge CLI	PySpark or SparkSQL (user selects)

T-SQL → Spark SQL Quick Mapping

T-SQL	Spark SQL
`GETDATE() / GETUTCDATE()`	`current_timestamp()`
`ISNULL(a, b) / NVL(a, b)`	`ifnull(a, b)`
`TOP N`	`LIMIT N`
`DATEADD(day, n, col)`	`date_add(col, n)`
`DATEDIFF(day, a, b)`	`datediff(b, a)`
`CONVERT(VARCHAR, col)`	`CAST(col AS STRING)`
`WITH (NOLOCK)`	`— removed (Delta MVCC)`
`MERGE ... WHEN MATCHED`	`Delta Lake MERGE INTO`
`SELECT INTO #temp`	`CREATE TEMP VIEW / CACHE TABLE`
`ROWNUM`	`ROW_NUMBER() OVER (ORDER BY ...)`